Question 6: Implement feature flags, perform load testing, and demonstrate Selenium-based automated testing; record commands, configurations, screenshots, and observations.

The app implements a FEATURE_BETA flag toggled by environment variable, apache bench generated 2000 requests of load, and a Selenium suite drove headless Chrome against the running app - all captured with logs and screenshots.

Feature flags

```
FEATURE_BETA = os.getenv("FEATURE_BETA", "false").lower() == "true"

@app.route("/beta")
def beta():
    if not FEATURE_BETA:
        return jsonify(feature="beta", enabled=False), 404
    return jsonify(feature="beta", enabled=True, message="Welcome to the beta feature")

$ docker run -d -e FEATURE_BETA=false devops-lab3-app:3.0.0
flag OFF: curl :5004/beta -> HTTP 404  {"enabled":false,...}

$ docker run -d -e FEATURE_BETA=true devops-lab3-app:3.0.0
flag ON:  curl :5004/beta -> HTTP 200  {"enabled":true,"message":"Welcome to the beta feature"}
```

Same image, different behavior - a feature can be toggled without a rebuild or a new deploy.

Load test (apache bench)

```
$ ab -n 2000 -c 50 http://localhost:5003/

Concurrency Level:      50
Complete requests:      2000
Failed requests:        0
Requests per second:    323.74 [#/sec] (mean)
Time per request:       154.446 [ms] (mean)
Time per request:       3.089 [ms] (mean, across all concurrent requests)

Percentage of the requests served within a certain time (ms)
  95%    211
  99%    267
```

Selenium automated browser testing

```
$ python tests/selenium_test.py http://localhost:5004
PASS  /health returned: {"status":"UP","version":"3.0.0"}
PASS  /beta flag enabled: {"enabled":true,"feature":"beta",...}
PASS  screenshot saved: docs/screenshots/q6-selenium-beta-page.png

$ chromedriver --version && google-chrome --version
ChromeDriver 157.0.8103.38
Google Chrome 157.0.8103.38
```

The Selenium suite runs headless Chrome: it asserts the /health payload, asserts the enabled beta flag and saves a browser screenshot - repeatable in any CI run.

Pretty-print ☐

{"enabled":true,"feature":"beta","message":"Welcome to the beta feature"}

Fig: Headless Chrome screenshot taken by the Selenium test

Observations

- Feature flags decouple deploy from release: the same image serves both variants.
- The load bottleneck is the Flask development server (323 req/s, p95 211 ms) - in production a WSGI server (gunicorn) would multiply this; the test pattern stays the same.
- Chrome and ChromeDriver versions must match exactly (157.0.8103.38 both) or the driver fails.
- Headless mode (--headless=new, --no-sandbox, --disable-dev-shm-usage) is required in containers/WSL.

Conclusion

All experiments were performed on Ubuntu 24.04 LTS with Ansible, Terraform, Prometheus, Grafana, the ELK stack and Selenium. The evidence (command logs, configuration files, screenshots and commit history) is committed in the GitHub repository github.com/theabhinav0231/devops-lab-3 under the docs/ directory.